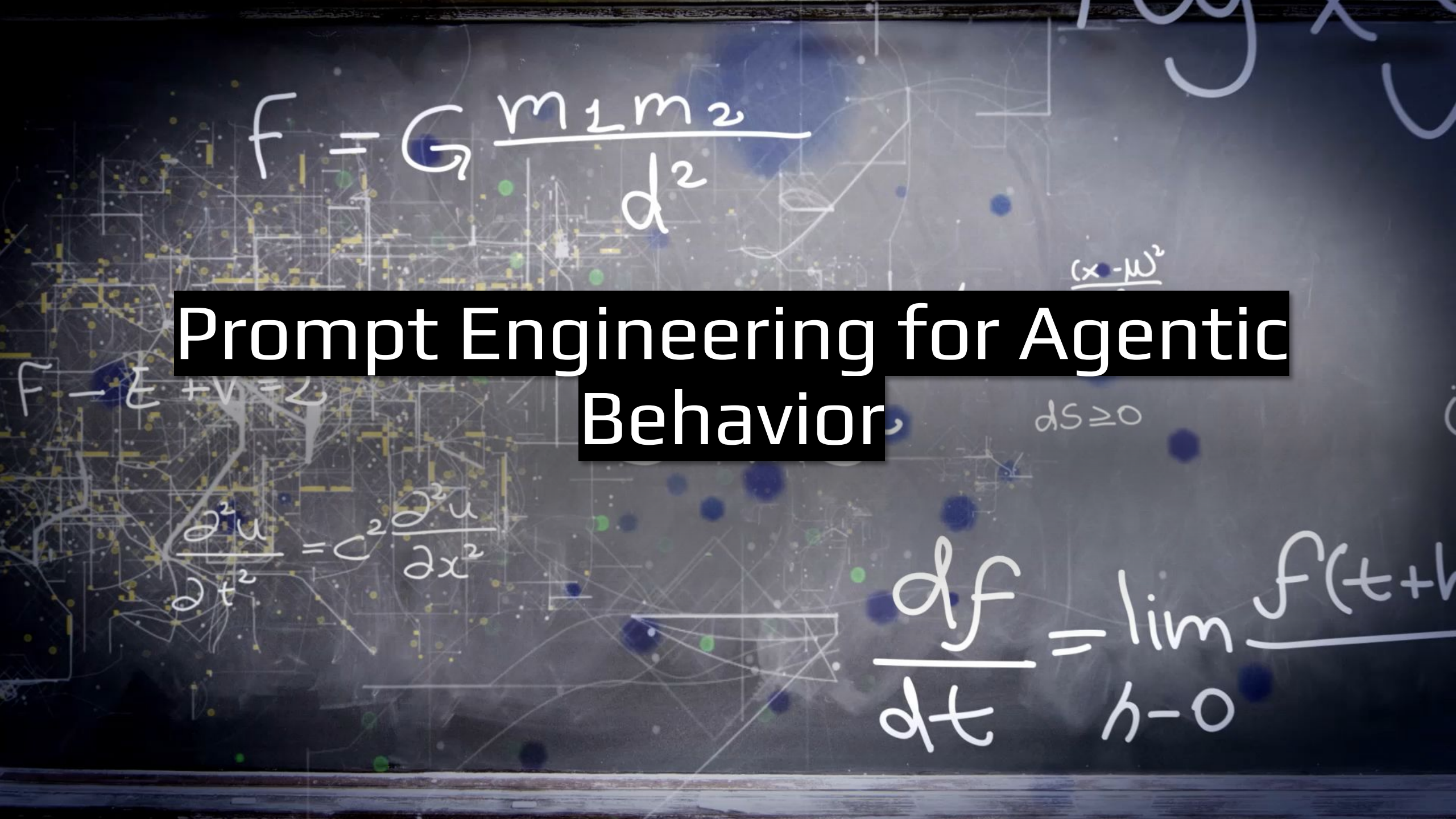
Prompt Engineering for Agentic Behavior

WHAT IS PROMPT ENGINEERING?

- Technique to design effective inputs for LLMs.
- Transforms raw models into structured reasoning agents.
- Guides reliability, accuracy, and task success.
- Acts as the "user interface" to AI reasoning.
- Foundation of Agentic AI behavior

❑Example

- ❑ **Prompt:** "*Summarize the article for a 12-year-old, highlighting causes and solutions in bullet form.*" → Clearer, structured, more useful.

ZERO-SHOT

- No examples provided
- Relies on model's pretraining data
- Good for well-known facts & simple queries

Example:

Prompt: "What is the capital of Japan?

"**Output:** "The capital of Japan is **Tokyo.**" (Reliable fact)

WHEN ZERO-SHOT WORKS WELL

- Questions with consistent data in training
- Common facts → “Paris is capital of France”
- Simple translations → “Hello → Hola”
- Arithmetic → “2+2=4”

Example:
Prompt: *“Translate: ‘How are you?’ into French.”*
Output: How are you?’ in French is **« Comment ça va ? »**

HALLUCINATIONS IN ZERO-SHOT

- LLMs don't "know" truth — only patterns
- Risk of confident but wrong answers
- Misleading prompts → hallucinations
- Example
- If you ask any LLM "What happened at Coca-Cola on March 17, 1978?",
- There is no notable public record of an event tied to that exact date.
  - LLM generate a confident story (e.g., "On March 17, 1978, Coca-Cola launched a new bottling plant in Atlanta" or "Coca-Cola introduced a marketing campaign in Europe"). Sound plausible but wrong, because nothing verifiable exists for that date.

FEW-SHOT PROMPTING

- In Few-Shot Prompting, we show the LLM examples of the task before asking the real question (Provide 2–5 exemplars before the query)
- These examples (called exemplars) act like a mini training set inside the prompt.
- If your exemplars are clear, the model imitates them correctly.
- If they're wrong, vague, or inconsistent → the model will copy those errors (→ hallucination risk).
- Best for classification, extraction, structured outputs
- Reduces ambiguity vs zero-shot, but not foolproof

EXAMPLE (SENTIMENT CLASSIFICATION)

Task: Classify text as {positive | negative}.

Example 1:

Text: "I loved the food!"Answer: positive

Example 2:

Text: "The service was terrible."Answer: negative

Now classify:

Text: "The ambience was nice but food was average.“

Answer:

CHAIN-OF-THOUGHT (COT)

Ask for step-by-step reasoning before answer

Improves multi-step math, logic, planning

Make reasoning concise & checkable

Example

- Solve step by step, then give a final answer key.
- A shop had 10 apples, sold 4, then bought 5 more.
- How many now?
- **Return:**
- - reasoning: brief steps
- - answer: integer

WHY HALLUCINATIONS INCREASE RISK IN COT

longer reasoning chains = more chances for error.

LLM tries to “fill in gaps” → fabricates intermediate facts.

a false step early → cascades into a wrong final answer.

- CoT expands token length
- LLM fills gaps when unsure
- confident fluency masks falsehoods

TOT EXAMPLE

Task: 1-day Lahore itinerary under PKR 5,000.

Step A (Propose 3 options):

- Option 1: ...

- Option 2: ...

- Option 3: ...

Step B (Score each):

- budget_ok (true/false), travel_time_ok, diversity_ok (cultural/food/parks)

Step C (Select best):- chosen_option: X

- justification: why scores are best

Return JSON: {options:[...], scores:[...], chosen_option, plan}

Outputs: Always **schema-constrained** (JSON) to reduce drift

ROLE & SYSTEM PROMPTING

- System prompts are like the constitution they set the overall rules for how an AI behaves.
- Role prompts are like job descriptions they define what the AI should do in a specific task.
- If we combine both, we get reliable and specialized behavior.
- For example, the system prompt might say: 'Always be neutral and polite,' while the role prompt says: 'You are a financial analyst.' The system ensures tone, while the role ensures domain expertise. In multi-agent systems, we often combine multiple roles under one shared system prompt."

MULTI-CONTEXT PROMPTING (MCP)

- Divide input into layered contexts
- Each context provides different types of information

Examples

Context A → Domain knowledge

Context B → Constraints (budget, rules, ethics)

Context C → Output style/format

Improves reasoning for complex tasks

EXAMPLE: RENEWABLE ENERGY PLAN

Context A (Domain Knowledge): Use information about air pollution in Lahore, the benefits of electric vehicles, and examples of successful adoption in other cities.

Context B (Constraints): Budget is limited to PKR 1 billion. Prioritize routes with the highest daily commuters. Ensure environmental sustainability is emphasized.

Context C (Output Style): Write in the form of a policy brief (50-70 words), formal tone, structured with headings: Background, Proposal, Benefits, Conclusion.

MCP REDUCES HALLUCINATIONS

- Keeps reasoning organized into buckets (knowledge, constraints, style)
- Reduces chance of model drifting into irrelevant details
- Forces traceable reasoning (each context maps to part of the answer)
- Useful in multi-agent workflows where agents handle different contexts

AUTOPROMPT

Technique where prompts are automatically generated

Uses search, optimization, or gradient signals to find effective prompts

Not always human-readable (may look strange)

Still highly effective for performance

- Instead of manually designing prompts…Algorithms search for tokens/triggers that maximize accuracy on a task
- Often uses reinforcement learning or optimization techniques
  - Example: Model learns that adding the token “!great” before text improves classification accuracy

EXAMPLE

For example, in sentiment classification, it might discover that adding ‘!good’ or ‘!bad’ improves accuracy.

These don’t mean much to humans, but they work for the model.

The challenge is that we lose interpretability it’s effective but not transparent.

REACT PROMPTING

- Combines Reasoning and Acting
- Loop: Think → Act → Observe → Reason → repeat

Example

You are an agent that can reason and act.

Step 1: Think about what info is needed.

Step 2: Act by calling the "search_population" tool.

Step 3: Observe the result.

Step 4: Reason again until you can answer.

REACT PREVENTS HALLUCINATIONS

Pure reasoning (like CoT) → model might hallucinate numbers

ReAct inserts tool calls for facts → grounding in real data

Each observation step anchors reasoning → less chance of making things up

LEAST-TO-MOST PROMPTING

- Break down complex problems into simpler sub-problems
- Tackle easy parts first, then harder steps
- Reduces cognitive load for the LLM
- Limits hallucinations by grounding on smaller correct steps
- Mimics how humans solve math, coding, or logic puzzles

EXAMPLE

Task: Solve $(27 + 53) \times (12 - 7)$

Step 1: Solve the smallest sub-problem: $27 + 53$.

Step 2: Solve $12 - 7$.

Step 3: Multiply results of Step 1 and Step 2.

SELF-ASK PROMPTIN G

- Model generates its own sub-questions before answering
- Encourages deeper reasoning and structured exploration
- Helps avoid skipping important steps
- Useful for causal analysis, reasoning, explanations
- LLMs often jump straight to an answer (risk: hallucination).

EXAMPLE

You are an AI that solves questions by generating your own sub-questions first.

Follow this process:

1. Read the main question.

2. Generate 2–4 sub-questions that break the problem into parts.

3. Answer each sub-question briefly.

4. Write a final synthesized answer.

Main Question: Why did the Roman Empire collapse?

META-PRO MPTING

Prompts that describe how to create other prompts

"Prompts about prompting"

Reduces human effort in prompt engineering.

- Meta-prompts may generate vague or weak prompts
- If not constrained, can lead to prompt drift (too generic or too verbose)

EXAMPLE

I want you to generate a prompt that will help an AI answer science questions more reliably.

The prompt should:

- Ask the AI to explain step-by-step (CoT)
- Request sources when available
- Limit answers to 150 words

SYMBOLIC REASONING PROMPTING

Encourage logical, step-by-step inference

Represent facts as symbols, equations, rules

Reduce ambiguity by using formal structure

EXAMPLE

Convert the following into symbolic logic and prove the conclusion.

Statement: All mammals are warm-blooded. Whales are mammals. Therefore, whales are warm-blooded.

Steps:1. Define symbols (predicates and constants).

2. List the premises in symbolic form.

3. Apply inference rules step by step.

4. Output the result as JSON only:

{ "answer": true/false, "justification": "..." }

PROGRAM-AIDED LANGUAGE (PAL)

- LLM writes code snippets to solve tasks
- Combines natural language reasoning + programming power
- Offloads complex computation to code execution
- Makes reasoning more reliable, less error-prone

EXAMPLE

Solve the problem using Python code.

Problem: What is $(27 + 53) \times (12 - 7)$?

Steps:

1. Write Python code to calculate.

2. Show the result.

Return JSON: { "code": "...", "answer": ... }

ITERATIVE PROMPTING

- Technique to refine outputs step by step
- Works like a feedback loop between user and model
- Instead of expecting perfection in one go → ask → review → refine
- Improves clarity, correctness, and reduces hallucinations

EXAMPLE

Round 1 – Initial Ask

Write a short explanation of black holes.

Round 2 – Refinement

Refine the explanation by:

- Expanding to 4–5 sentences

- Including how black holes are formed

-Keeping it simple for a high school student

Round 3 – Final Refinement

Improve the explanation again by:

- Adding an example of a famous black hole

- Explaining what the event horizon is

- Keeping the language clear and concise

SEQUENTIAL PROMPTING

- Technique to chain multiple prompts in sequence
- Each response becomes input for the next step
- Useful for multi-step workflows (research → summarize → analyze → present)
- Common in agent frameworks (LangChain, CrewAI, LangGraph)

EXAMPLE

Prompt 1

List 3 main causes of climate change.

Prompt 2

Take the 3 causes you just listed and write a short paragraph explaining them in simple language for high school students.

Prompt 3

Now, based on that paragraph, write a 2-line conclusion that encourages students to take action against climate change.

SELF-CONSISTENCY PROMPTING

- Extension of Chain-of-Thought (CoT).
- Instead of generating one reasoning chain, the LLM generates multiple independent reasoning chains.
- The final answer is chosen by majority vote / most consistent result.
- Goal = reduce random hallucinations by checking consistency across multiple runs.

SCP-Analogy: Ask 10 students to solve the same math problem separately → trust the majority answer.

ToT-Analogy: A team brainstorms options on a whiteboard, branches out ideas, evaluates them, prunes weak ones, and follows the best path.

CHAIN-OF-THOUGHT WITH MULTIPLE ATTEMPTS

Solve this problem using step-by-step reasoning.

Problem: A farmer has 25 chickens. He sells 7 of them and then buys 12 more.

How many chickens does the farmer have now?

Instructions:

1. Generate 3 different step-by-step reasoning paths independently.

2. Show the final answer for each path.

3. Identify the most common (majority) answer.

4. Report that as the final result.

Return your answer in JSON:

```
{  "paths": [
    {"steps": "...", "answer": ...},
    {"steps": "...", "answer": ...},
    {"steps": "...", "answer": ...}
  ],  "final_result": ... }
```

GENERATED KNOWLEDGE PROMPTING

- The model generates extra knowledge first → then uses it to solve the task
- Creates a temporary fact base before answering
- Makes reasoning more accurate & grounded
- Useful when task requires background knowledge that isn't obvious

EXAMPLE

Step 1: Generate 3–4 facts about the causes of World War I.

Step 2: Using those facts, explain why the war started.

Return the final explanation in 4–5 sentences.

HANDLING HALLUCINATIONS IN PROMPTING

- Ask for **sources:** “Cite references for your answer”
- Use **Self-Consistency**: query multiple times, compare answers
- Apply **Iterative Prompting**: refine question step by step
- Combine with **Retrieval-Augmented Generation (RAG)**: pull facts from external database
- **Example (fixing):**
  Prompt: “*Where was Quaid-e-Azam Muhammad Ali Jinnah born? Provide reliable source.*”
  Output: “*He was born in Karachi, 1876 (Ref: Wikipedia).*”